

LABORATORIO #3

ARSW

PRESENTADO A: DIEGO ANDRÉS TRIVIÑO GONZALEZ

REALIZADO POR:

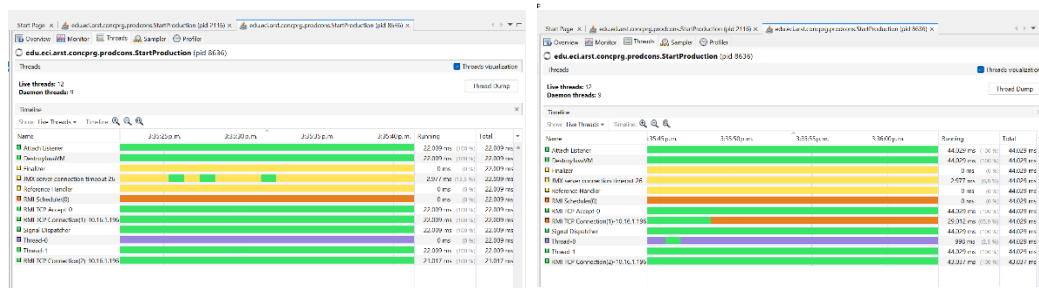
MARIANA MALAGÓN

PAULA LOZANO

14/08/2026



UNIVERSIDAD



2. *Haga los ajustes necesarios para que la solución use más eficientemente la CPU, teniendo en cuenta que -por ahora- la producción es lenta y el consumo es rápido. Verifique con JVisualVM que el consumo de CPU se reduzca.*

```
@Override
public void run() {
    while (true) {
        synchronized (queue) {
            while (queue.isEmpty()) {
                try {
                    queue.wait();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    return;
                }
            }

            int elem=queue.poll();
            System.out.println("Consumer consumes "+elem);
        }
    }
}
```

Para el hilo consumer, como tiene que mantener su consumo rápido, entonces utilizamos la función .wait() sobre el objeto que tiene en el lock, de manera que si encuentra la cola vacía no realiza ninguna acción y suelta el lock sobre el objeto para que el hilo producer pueda almacenar objetos en la cola.

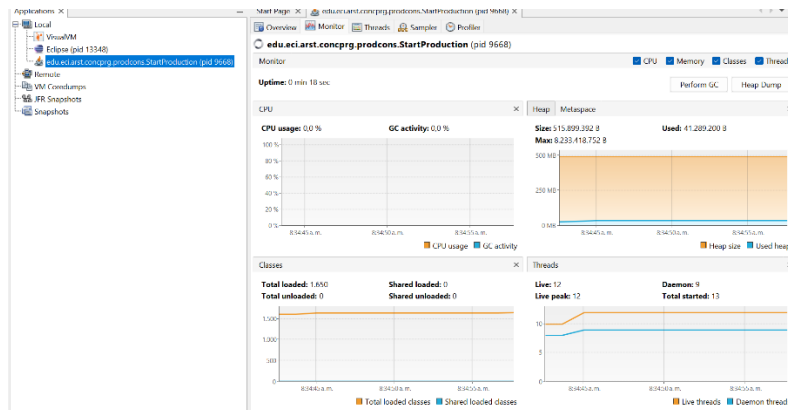
```
@Override
public void run() {
    while (true) {
        synchronized (queue) {
            dataSeed = dataSeed + rand.nextInt(100);
            System.out.println("Producer added " + dataSeed);
            queue.add(dataSeed);
            queue.notify();
        }
        try {
            Thread.sleep(1000);
        } catch (InterruptedException ex) {
            Logger.getLogger(Producer.class.getName()).log(Level.SEVERE, null, ex);
        }
    }
}
```

Con el hilo producer mantuvimos la estrategia y con la función .notify() avisamos al hilo consumer que ya puede continuar con su trabajo. Utilizamos solo notify() en lugar de notifyAll() porque tenemos un único hilo.

Observamos los nuevos resultados:

Running Sleeping Wait Park Monitor





El uso de CPU bajó completamente y vemos que el thread consumer no está en runnable el 100% del tiempo (que era el color verde), sino que está en estado wait (color amarillo).

3. *Haga que ahora el productor produzca muy rápido, y el consumidor consuma lento. Teniendo en cuenta que el productor conoce un límite de Stock (cuantos elementos debería tener, a lo sumo en la cola), haga que dicho límite se respete. Revise el API de la colección usada como cola para ver cómo garantizar que dicho límite no se supere. Verifique que, al poner un límite pequeño para el 'stock', no haya consumo alto de CPU ni errores.*

```
@Override
public void run() {
    while (true) {
        synchronized (queue) {
            while (queue.size() >= stockLimit) {
                try {
                    queue.wait();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    return;
                }
            }
            dataSeed = dataSeed + rand.nextInt(100);
            System.out.println("Producer added " + dataSeed);
            queue.add(dataSeed);
            queue.notifyAll();
        }
    }
}
```

Ahora el hilo Producer está en constante producción porque eliminamos el `.sleep()`. Sin embargo, verificamos que lo haga siempre y cuando la cola no supere el límite de objetos que puede tener. Apenas incluye un nuevo objeto, notificamos con `notifyAll()`, porque puede que el hilo Consumer esté esperando para poder ejecutarse.

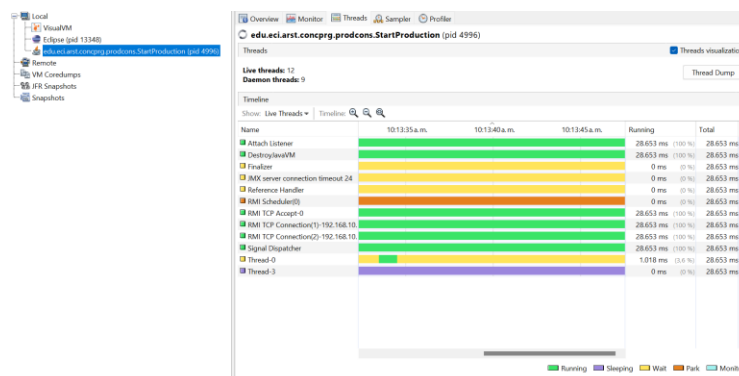
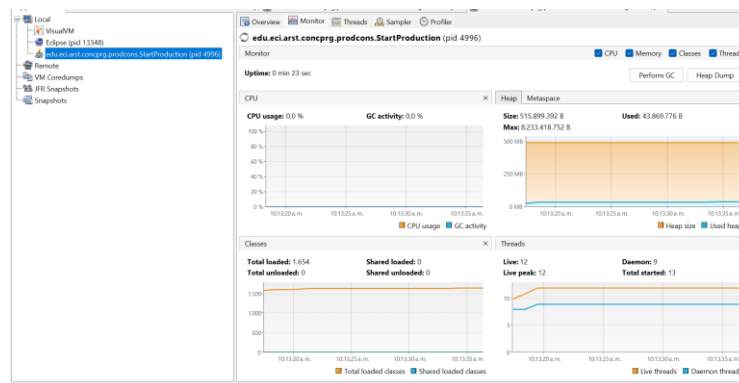
```

@Override
public void run() {
    while (true) {
        synchronized (queue) {
            while (queue.isEmpty()) {
                try {
                    queue.wait();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    return;
                }
            }
            int elem=queue.poll();
            System.out.println("Consumer consumes "+elem);
            queue.notifyAll();
        }
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

```

Por el lado del hilo Consumer, utilizamos `.sleep(1000)` (1000 milisegundos, es decir, 1 segundo) para que el consumo fuera lento.

En el main, el hilo Producer se crea con `stockLimit=5`. Con esto verificamos el comportamiento en VisualVM:



El uso de CPU se mantiene casi nulo. Vemos que el hilo Producer se encuentra en estado waiting, porque llena la cola rápidamente hasta el límite y luego queda bloqueado esperando a que el Consumer libere espacio (llamando `queue.wait()`, que bloquea indefinidamente hasta recibir una notificación). El hilo Consumer, en cambio,

se encuentra en estado sleeping, porque está ejecutando Thread.sleep(1000) mientras cuenta ese segundo antes de volver a consumir.

Decidimos usar notifyAll() en lugar de notify() en ambas clases. Con un solo Producer y un solo Consumer, notify() habría sido suficiente, ya que solo puede haber un hilo del otro tipo esperando en el monitor de queue en un momento dado, así que no existe ambigüedad sobre a quién despertar.

Sin embargo, si el sistema tuviera varios hilos Producer y/o varios Consumer compartiendo la misma cola, notify() se volvería riesgoso: este método despierta a **un solo hilo elegido arbitrariamente** entre todos los que estén esperando en el monitor, sin distinguir si ese hilo esperaba porque la cola estaba vacía o porque estaba llena. Esto podría llevar a que, por ejemplo, se despierte a otro Producer cuando en realidad era un Consumer quien necesitaba reaccionar (o viceversa), dejando a ese hilo bloqueado innecesariamente mientras el que sí podía avanzar sigue esperando. Ese escenario puede derivar en hilos que se quedan esperando indefinidamente aunque técnicamente había condiciones para que avanzaran, afectando el rendimiento del sistema.

Parte II

Para esta parte, trabajaremos sobre el repositorio que entregamos en el último laboratorio sobre una nueva rama que llamaremos: feature/lab2-partII:

https://github.com/marianamalagon11/ARSW_PARALLELISM_THREADS/tree/feature/lab2-partII

En el README.md colocamos toda la solución.

Parte III

1. *Revise el programa “highlander-simulator”, dispuesto en el paquete edu.eci.arsw.highlandersim. Este es un juego en el que:*
 - a. *Se tienen N jugadores inmortales.*
 - b. *Cada jugador conoce a los N-1 jugador restantes.*
 - c. *Cada jugador, permanentemente, ataca a algún otro inmortal. El que primero ataca le resta M puntos de vida a su contrincante, y aumenta en esta misma cantidad sus propios puntos de vida.*
 - d. *El juego podría nunca tener un único ganador. Lo más probable es que al final sólo queden dos, peleando indefinidamente quitando y sumando puntos de vida.*

2. *Revise el código e identifique cómo se implementó la funcionalidad antes indicada.*

La funcionalidad descrita en el punto 1, se implementa principalmente en la clase `Immortal`, que extiende `Thread`, y cada instancia representa a un jugador inmortal corriendo en su propio hilo.

N jugadores: existe una lista llamada `immortalPopulation` con una cantidad `N` de `Immortals`.

Cada jugador conoce a los N-1 jugadores restantes: cada inmortal conoce a los demás a través de una lista compartida (`immortalsPopulation`), pasada por constructor a todos los hilos.

Ciclo permanente de combate: el método `run()` contiene un `while(true)` que nunca termina, lo que garantiza que cada inmortal esté "permanentemente" atacando, tal como pide el enunciado. En cada vuelta del ciclo, el inmortal ejecuta tres pasos: elige un oponente, lo ataca, y hace una pequeñísima pausa (`Thread.sleep(1)`) antes de repetir.

Elección del oponente: Para elegir a quién atacar, se genera un índice aleatorio dentro del rango de la lista (`r.nextInt(immortalsPopulation.size())`). Se evita el auto-ataque comparando ese índice contra la posición propia; si coinciden, se pasa al índice inmediatamente siguiente de modo que ningún inmortal se ataque a sí mismo.

El ataque (`fight()`): el método implementa la mecánica central: si el oponente `i2` tiene salud mayor a 0, se le restan `defaultDamageValue` puntos, y esa misma cantidad se le suma al atacante (`this`). Vemos que los puntos no se "crean" ni se "destruyen", simplemente se transfieren de un jugador a otro. Si el oponente ya tiene salud igual o menor a 0 (ya está "muerto"), no ocurre ninguna transferencia de puntos, solo se registra un mensaje indicando que el intento de ataque fue contra un inmortal ya derrotado.

Dada la intención del juego, un invariante debería ser que la sumatoria de los puntos de vida de todos los jugadores siempre sea el mismo (claro está, en un instante de tiempo en el que no esté en proceso una operación de incremento/reducción de tiempo). Para este caso, para N jugadores, ¿cuál debería ser este valor?

Para `N` jugadores, el valor del invariante debería ser $N \times 100$, dado que cada inmortal inicia con una salud de 100 puntos:
`DEFAULT_IMMORTAL_HEALTH = 100`, definida en `ControlFrame`.

3. *Ejecute la aplicación y verifique cómo funcionan la opción 'pause and check'. ¿Se cumple el invariante?*

El enunciado dice que el invariante debería cumplirse "en un instante donde no haya una operación de incremento/reducción en curso", y ahí está la clave de todo. Solo se mantiene el invariante si se mira el sistema entre peleas completas, no a la mitad de una.

El problema es que al correr la simulación con $N=3$ (donde debería dar siempre 300) y darle varias veces a "Pause and check", los resultados fueron: 1010, 4510, 5300, 6020, 6480. Es una suma que crece sin parar, lo que dice que el sistema está "ganando" puntos de la nada todo el tiempo, no que de vez en cuando se lee mal.

La causa está en `fight()`: resta salud al oponente y luego suma esa misma cantidad al atacante, pero son dos operaciones separadas y ninguna es atómica. La resta en realidad son tres pasos escondidos (leer salud, calcular, escribir), así que si dos inmortales atacan al mismo objetivo casi al mismo tiempo, ambos pueden leer el mismo valor antes de que el otro alcance a escribir.

Verificando la funcionalidad del botón "Pause and check", en realidad no pausa nada todavía: solo recorre la lista y suma la salud mientras los demás hilos siguen peleando en segundo plano.

4. *Una primera hipótesis para que se presente la condición de carrera para dicha función (pause and check), es que el programa consulta la lista cuyos valores va a imprimir, a la vez que otros hilos modifican sus valores. Para corregir esto, haga lo que sea necesario para que efectivamente, antes de imprimir los resultados actuales, se pausen todos los demás hilos. Adicionalmente, implemente la opción 'resume'.*

Inicialmente resolvimos este punto usando las funciones `suspend()` y `resume()` heredadas de `Thread`, ya que a primera vista parecían justamente las que buscábamos para "pausar" y "continuar" hilos. Sin embargo, al seguir trabajando en el laboratorio (punto 7) descubrimos que esta API está deprecada precisamente porque puede congelar un hilo en cualquier punto de su ejecución, incluso mientras tiene locks tomados, lo cual nos generó un problema de bloqueo no determinista.

Por eso reemplazamos esa estrategia por una de autopausa cooperativa: cada `Immortal` tiene una bandera `suspended` que él mismo revisa al inicio de cada vuelta del `while(true)`, antes de llamar a `fight()`, es decir que en el único punto donde el hilo no tiene ningún lock tomado. Si la bandera está activa, el hilo se bloquea con `wait()` sobre un lock propio (`pauseLock`), liberando cualquier recurso, hasta que `resumeImmortal()` lo despierta con `notifyAll()`.

Adicionalmente, para que "Pause and check" no lea la salud de los inmortales mientras alguno todavía está terminando una pelea, agregamos una clase `PauseCoordinator` compartida: cada inmortal le avisa cuando efectivamente llegó al punto de pausa, y el botón espera (`awaitAllPaused()`) a que todos hayan confirmado antes de sumar la salud. Este mecanismo usa exactamente el mismo patrón `synchronized/wait/notify` que ya habíamos usado en `Productor/Consumidor` (Parte I), solo que coordinando un contador en vez de una cola.

```
private void checkSuspension() {
    if (suspended) {
        coordinator.threadPaused();
        synchronized (pauseLock) {
            while (suspended) {
                try {
                    pauseLock.wait();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    return;
                }
            }
        }
        coordinator.threadResumed();
    }
}
```

```
public void pauseImmortal() {
    suspended = true;
}

public void resumeImmortal() {
    synchronized (pauseLock) {
        suspended = false;
        pauseLock.notifyAll();
    }
}
```

```
JButton btnPauseAndCheck = new JButton(text: "Pause and check");
btnPauseAndCheck.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        for (Immortal im : immortals) {
            im.pauseImmortal();
        }

        try {
            pauseCoordinator.awaitAllPaused();
        } catch (InterruptedException ex) {
            Thread.currentThread().interrupt();
        }
    }
});
```

5. *Verifique nuevamente el funcionamiento (haga clic muchas veces en el botón). ¿Se cumple o no el invariante?*

Nuevamente corrimos el programa, y aunque pudimos evidenciar que con el botón "Pause and check" los hilos se detienen (dejan de imprimirse resultados de las

interacciones) y con "Resume" continúan ejecutando sus tareas con normalidad, el invariante sigue sin cumplirse; el "Health sum" continúa creciendo entre una verificación y otra.

Suspender los hilos antes de sumar sí solucionó el problema de leer el sistema a medio actualizar, pero no corrige el problema de fondo: el método `fight()` sigue sin ser atómico.

6. *Identifique posibles regiones críticas en lo que respecta a la pelea de los inmortales. Implemente una estrategia de bloqueo que evite las condiciones de carrera. Recuerde que, si usted requiere usar dos o más 'locks' simultáneamente, puede usar bloques sincronizados anidados.*

La región crítica de la simulación está en el método `fight()` de la clase `Immortal`. Ahí es donde se leen y modifican los valores de salud de dos objetos distintos (el atacante `this` y el atacado `i2`) sin ningún tipo de "protección". Como múltiples hilos pueden ejecutar `fight()` simultáneamente sobre los mismos inmortales, estas lecturas y escrituras concurrentes son la causa de la condición de carrera identificada en los puntos anteriores: una resta de salud puede perderse mientras la suma correspondiente al atacante sí se aplica.

Para corregir esto, sincronizamos el método `fight()` sobre los dos objetos involucrados en cada pelea, usando bloques `synchronized` anidados, tal como sugiere el enunciado. Sin embargo, antes de implementarlo de esta forma, tuvimos en cuenta una advertencia explicada por el profesor en clase: el uso de bloques `synchronized` anidados sobre dos objetos es una causa común de deadlock cuando el orden en que se toman los locks no es consistente entre los distintos hilos. En este caso particular, si dos inmortales se atacan mutuamente casi al mismo tiempo (por ejemplo, `im0` ataca a `im1` mientras `im1` ataca a `im0`), cada hilo tomaría primero el lock de `this` y luego intentaría tomar el de `i2`, pero como el "atacante" y el "atacado" se invierten según el hilo, cada uno terminaría esperando el lock que el otro ya tiene tomado, generando una espera circular.

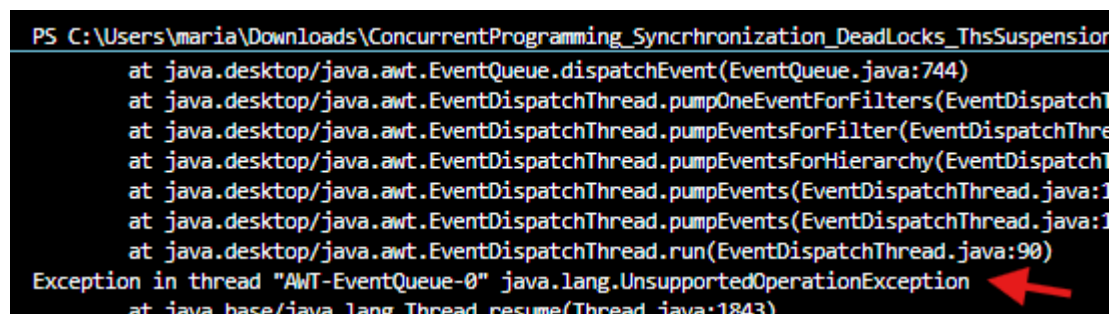
Para prevenir esto desde el principio, decidimos implementar de una vez una estrategia de orden consistente de adquisición de locks, que aprendimos en clase, en lugar de sincronizar directamente sobre `this` e `i2` en ese orden fijo. Reforzamos este tema revisando previamente las páginas 206 y 207 de *Java Concurrency in Practice*, donde se explica esta misma técnica (sección 10.1.2, "Dynamic lock order deadlocks"). De las opciones posibles, optamos por usar el `hashCode` de cada objeto como criterio de orden, por ser la forma más sencilla y rápida de implementar: en lugar de depender de quién ataca a quién, se determina de antemano cuál de los dos inmortales debe bloquearse primero según ese criterio fijo, de modo que todos los hilos, sin importar el orden en que ejecuten `fight()`, intenten tomar los locks en la misma secuencia.

7. *Tras implementar su estrategia, ponga a correr su programa, y ponga atención a si éste se llega a detener. Si es así, use los programas jps y jstack para identificar por qué el programa se detuvo.*

Al ejecutar la simulación tras implementar la estrategia de bloqueo con orden consistente en `fight()` (punto 6), intentamos provocar y observar el bloqueo esperado en "Pause and check"/"Resume". Conceptualmente, el problema de usar `Thread.suspend()` es que congela un hilo en el punto exacto de su ejecución en el que recibe la orden, sin importar qué locks tenga tomados. Como `fight()` toma dos locks anidados (`synchronized(first){ synchronized(second){...} }`), si la suspensión cae justo ahí, el hilo queda congelado sosteniendo esos locks, y cualquier otro inmortal que necesite pelear contra alguno de los dos quedaría bloqueado esperando un lock que nadie soltaría hasta el `resume()`, el clásico deadlock de `Thread.suspend()` descrito en JCiP.

Sin embargo, al intentar reproducirlo en la práctica para capturarlo con `jstack`, encontramos algo distinto: cada clic en "Pause and check" o "Resume" lanzaba inmediatamente `java.lang.UnsupportedOperationException` en el hilo `AWT-EventQueue-0`, tanto en `Thread.suspend()` como en `Thread.resume()`. Investigando la causa, confirmamos que estamos usando Java 21, donde estos métodos ya no solo están deprecados: su implementación fue reemplazada para lanzar esta excepción incondicionalmente cada vez que se invocan. Es decir, el propio JDK bloquea por completo el uso de esta API, precisamente por el riesgo de deadlock que acabamos de describir.

Esto explica por qué nunca observamos la simulación "congelarse" visualmente: los hilos nunca llegaron a suspenderse realmente, porque la llamada fallaba antes de tener efecto. No fue necesario usar `jps/jstack` para diagnosticar un bloqueo en tiempo de ejecución, ya que el JDK expone el problema directamente en la traza de la excepción.



```
PS C:\Users\maria\Downloads\ConcurrentProgramming_Synchronization_DeadLocks_ThsSuspension>
at java.desktop/java.awt.EventQueue.dispatchEvent(EventQueue.java:744)
at java.desktop/java.awt.EventDispatchThread.pumpOneEventForFilters(EventDispatchThread.java:209)
at java.desktop/java.awt.EventDispatchThread.pumpEventsForFilter(EventDispatchThread.java:226)
at java.desktop/java.awt.EventDispatchThread.pumpEventsForHierarchy(EventDispatchThread.java:241)
at java.desktop/java.awt.EventDispatchThread.pumpEvents(EventDispatchThread.java:255)
at java.desktop/java.awt.EventDispatchThread.pumpEvents(EventDispatchThread.java:249)
at java.desktop/java.awt.EventDispatchThread.run(EventDispatchThread.java:90)
Exception in thread "AWT-EventQueue-0" java.lang.UnsupportedOperationException
at java.base/java.lang.Thread.resume(Thread.java:1843)
```

8. *Plantee una estrategia para corregir el problema antes identificado (puede revisar de nuevo las páginas 206 y 207 de Java Concurrency in Practice).*

La técnica de las transferencias bancarias que explicó el profesor en clase, basada en un orden consistente de adquisición de locks, es la misma que ya habíamos aplicado en el punto 6 para resolver el deadlock dentro de `fight()`. Sin embargo, esa técnica

no resuelve el problema descrito en el punto 7, que es de naturaleza distinta: no se trata de un mal orden de adquisición de locks, sino de que Thread.suspend()/resume() pueden congelar (o, en nuestro caso, directamente impedir la ejecución) sin darle al hilo ninguna oportunidad de soltar los locks que tiene tomados.

Para este problema puntual, revisamos nuevamente las páginas 206-207 de Java Concurrency in Practice, donde se recomienda no usar nunca suspend()/resume()/stop(), y en su lugar implementar la pausa de forma cooperativa: el propio hilo se revisa a sí mismo en un punto seguro de su código, fuera de cualquier sección crítica, y decide si debe esperar. Implementamos esto con una bandera (suspended) revisada al inicio del ciclo del run(), antes de entrar a fight(), usando wait()/notifyAll() sobre un lock dedicado a la pausa, distinto del lock usado para pelear. Como la revisión ocurre siempre fuera de la región crítica de fight(), es imposible que un hilo quede "congelado" sosteniendo un lock de pelea, el problema queda eliminado de raíz, sin importar en qué JDK se ejecute.

9. *Una vez corregido el problema, rectifique que el programa siga funcionando de manera consistente cuando se ejecutan 100, 1000 o 10000 inmortales. Si en estos casos grandes se empieza a incumplir de nuevo el invariante, debe analizar lo realizado en el paso 4.*

Con 100:

```
Start Pause and check Resume num. of immortals: 100 STOP
imm33[50] says:im18[0] is already dead!
Fight: im2[10] vs im5[0]
12[0] is already dead!
ready dead!
ys:im14[0] says:im72[0] is already dead!
m56[0] is already dead!
im53[10] says:im24[0] is already dead!
im7[0] saim69[0] says:im52[0] is already dead!
im92[120] says:im81[0] isFight: im35[10] vs im1[10]
already deaim13[0] says:im55[0] is already dead!
dlim4[0] says:im18[0] is already dead!
Fight: im77[110] vs im2[0]

im91[300] says:im97[10] says:im34[0] is already dead!
im34[0] is already dead!
ym49[330] says:im29[0] Fight: im70[1310] vs im33[40]
is already dead!
sim34[0] says:im81[im64[0] says:im52[0]Fight: im83[30] vs im35[0]
im36[0] says:im13[0] is already dead!
im26[0] says:im55[0] is already dead!
j is already dead!
0 is already deadim48[0] says:im29[0] is already dead!
!
im37[0] iim31[0] says:im29[0] is already dead!
s already dead!
imim18[0] says:im19[0] is already dead!
56[0] saim88[0] says:im56[0] is already dead!
ys:im56[0] is already dead!
im66[670] says:im18[0] is already dead!
im25[30] says:im4[0] is already dead!
im11[0] says:im13[0] is already dead!
im93[10] says:im4[0] is already dead!
im85[770] says:im4[0] is already dead!
im3[0] says:im4[0] is already dead!
im16[60] says:im4[0] is already dead!

[im0[0], im1[10], im2[0], im3[0], im4[0], im5[280], im6[0], im7[0], im8[0], im9[210], im10[80], im11[0], im12[0], im13[0], im14[0], im15[0], im16[60], im17[0], im18[0], im19[0], im20[550], im21[0], im22[0], im23[0], im24[0], im25[30], im26[0], im27[10], im28[0], im29[0], im30[0], im31[0], im32[0], im33[40], im34[0], im35[0], im36[0], im37[0], im38[0], im39[10], im40[0], im41[370], im42[0], im43[0], im44[0], im45[230], im46[640], im47[620], im48[0], im49[330], im50[30], im51[130], im52[0], im53[10], im54[60], im55[0], im56[0], im57[0], im58[0], im59[0], im60[10], im61[0], im62[10], im63[1320], im64[0], im65[10], im66[670], im67[60], im68[0], im69[0], im70[1310], im71[0], im72[0], im73[10], im74[100], im75[100], im76[0], im77[110], im78[10], im79[30], im80[10], im81[0], im82[0], im83[30], im84[0], im85[770], im86[0], im87[60], im88[0], im89[240], im90[0], im91[300], im92[120], im93[10], im94[320], im95[10], im96[10], im97[10], im98[580], im99[80]]
Health sum:10000
```

Con 1000:

	StartPause and checkResume num. of immortals: 1000 STOP
im141[2040], im142[0], im143[0], im144[10], im145[0], im146[40], im147[210], im148[0], im149[10], im150[980], im151[0], im152[0], im153[0], im154[0], im155[0], im156[760], im157[0], im158[990], im159[0], im160[0], im161[40], im162[40], im163[0], im164[0], im165[0], im166[310], im167[0], im168[20], im169[10], im170[430], im171[0], im172[140], im173[0], im174[0], im175[0], im176[30], im177[410], im178[60], im179[0], im180[0], im181[30], im182[0], im183[0], im184[0], im185[0], im186[0], im187[160], im188[0], im189[0], im190[0], im191[420], im192[0], im193[10], im194[0], im195[550], im196[0], im197[20], im198[0], im199[0], im200[300], im201[0], im202[02740], im203[0], im204[0], im205[0], im206[1010], im207[0], im208[0], im209[30], im210[0], im211[1190], im212[160], im213[0], im214[10], im215[10], im216[0], im217[0], im218[0], im219[0], im220[0], im221[70], im222[0], im223[0], im224[0], im225[10], im226[10], im227[0], im228[0], im229[900], im230[520], im231[0], im232[0], im233[0], im234[10], im235[0], im236[10], im237[0], im238[0], im239[0], im240[0], im241[120], im242[100], im243[20], im244[560], im245[0], im246[0], im247[110], im248[350], im249[0], im250[0], im251[10], im252[0], im253[0], im254[0], im255[0], im256[0], im257[0], im258[0], im259[0], im260[10], im261[0], im262[0], im263[440], im264[0], im265[0], im266[0], im267[0], im268[0], im269[0], im270[0], im271[0], im272[100], im273[0], im274[0], im275[0], im276[20], im277[970], im278[90], im279[30], im280[0], im281[440], im282[0], im283[0], im284[10], im285[0], im286[0], im287[10], im288[900], im289[0], im290[0], im291[680], im292[0], im293[0], im294[0], im295[10], im296[0], im297[20], im298[0], im299[0], im300[0], im301[10], im302[30], im303[0], im304[0], im305[0], im306[0], im307[0], im308[0], im309[20], im310[0], im311[290], im312[10], im313[0], im314[20], im315[0], im316[0], im317[0], im318[10], im319[390], im320[10], im321[0], im322[0], im323[0], im324[0], im325[0], im326[10], im327[10], im328[0], im329[1220], im330[90], im331[410], im332[0], im333[0], im334[30], im335[0], im336[0], im337[580], im338[0], im339[0], im340[490], im341[50], im342[170], im343[0], im344[0], im345[0], im346[220], im347[430], im348[0], im349[10], im350[0], im351[0], im352[0], im353[0], im354[0], im355[0], im356[30], im357[10], im358[10], im359[0], im360[0], im361[10], im362[0], im363[0], im364[0], im365[30], im366[360], im367[0], im368[0], im369[0], im370[20], im371[60], im372[80], im373[0], im374[60], im375[0], im376[0], im377[10], im378[0], im379[560], im380[20], im381[0], im382[10], im383[760], im384[0], im385[40], im386[0], im387[10], im388[10], im389[10], im390[0], im391[1460], im392[0], im393[1170], im394[0], im395[280], im396[50], im397[70], im398[10], im399[0], im400[870], im401[0], im402[170], im403[0], im404[60], im405[0], im406[190], im407[0], im408[0], im409[0], im410[0], im411[1430], im412[30], im413[0], im414[0], im415[80], im416[0], im417[10], im418[0], im419[130], im420[60], im421[0], im422[30], im423[0], im424[60], im425[0], im426[30], im427[60], im428[50], im429[0], im430[100], im431[0], im432[130], im433[0], im434[0], im435[500], im436[30], im437[0], im438[0], im439[30], im440[10], im441[90], im442[90], im443[20], im444[160], im445[30], im446[0], im447[10], im448[0], im449[20], im450[0], im451[0], im452[0], im453[0], im454[90], im455[1480], im456[0], im457[0], im458[0], im459[0], im460[0], im461[130], im462[10], im463[0], im464[0], im465[0], im466[0], im467[0], im468[50], im469[670], im470[970], im471[10], im472[20], im473[50], im474[10], im475[0], im476[0], im477[330], im478[160], im479[10], im480[0], im481[230], im482[0], im483[0], im484[0], im485[10], im486[0], im487[340], im488[40], im489[0], im490[980], im491[300], im492[0], im493[0], im494[0], im495[0], im496[0], im497[0], im498[0], im499[0], im500[0], im501[1290], im502[0], im503[0], im504[10], im505[10], im506[0], im507[0], im508[0], im509[0], im510[0], im511[0], im512[0], im513[0], im514[0], im515[0], im516[0], im517[0], im518[10], im519[0], im520[10], im521[20], im522[0], im523[0], im524[0], im525[0], im526[0], im527[530], im528[10], im529[0], im530[0], im531[1250], im532[10], im533[20], im534[0], im535[0], im536[1080], im537[160], im538[0], im539[20], im540[0], im541[630], im542[0], im543[340], im544[0], im545[430], im546[700], im547[0], im548[0], im549[50], im550[130], im551[210], im552[0], im553[0], im554[0], im555[0], im556[1370], im557[1170], im558[0], im559[2370], im560[10], im561[430], im562[0], im563[0], im564[0], im565[10], im566[0], im567[0], im568[250], im569[0], im570[40], im571[0], im572[170], im573[20], im574[0], im575[0], im576[0], im577[0], im578[0], im579[0], im580[160], im581[60], im582[650], im583[230], im584[30], im585[0], im586[0], im587[40], im588[210], im589[0], im590[0], im591[360], im592[0], im593[180], im594[0], im595[0], im596[0], im597[0], im598[0], im599[270], im600[0], im601[0], im602[0], im603[0], im604[0], im605[0], im606[0], im607[10], im608[30], im609[0], im610[0], im611[20], im612[0], im613[0], im614[0], im615[0], im616[0], im617[0], im618[0], im619[0], im620[20], im621[50], im622[10], im623[0], im624[170], im625[0], im626[0], im627[300], im628[40], im629[0], im630[0], im631[0], im632[0], im633[0], im634[0], im635[110], im636[0], im637[50], im638[0], im639[0], im640[220], im641[0], im642[0], im643[0], im644[0], im645[0], im646[0], im647[10], im648[0], im649[0], im650[0], im651[0], im652[0], im653[0], im654[0], im655[0], im656[0], im657[10], im658[0], im659[0], im660[0], im661[0], im662[0], im663[210], im664[0], im665[0], im666[0], im667[0], im668[0], im669[0], im670[0], im671[0], im672[10], im673[0], im674[0], im675[0], im676[0], im677[0], im678[0], im679[0], im680[220], im681[270], im682[0], im683[0], im684[0], im685[0], im686[50], im687[0], im688[10], im689[0], im690[10], im691[0], im692[0], im693[50], im694[420], im695[5760], im696[40], im697[0], im698[0], im699[10], im700[0], im701[0], im702[0], im703[0], im704[0], im705[0], im706[0], im707[0], im708[0], im709[220], im710[0], im711[0], im712[270], im713[0], im714[0], im715[0], im716[160], im717[90], im718[20], im719[0], im720[0], im721[10], im722[0], im723[120], im724[0], im725[0], im726[0], im727[0], im728[0], im729[0], im730[0], im731[0], im732[270], im733[0], im734[0], im735[0], im736[0], im737[0], im738[70], im739[0], im740[0], im741[0], im742[0], im743[80], im744[20], im745[0], im746[0], im747[20], im748[0], im749[30], im750[10], im751[50], im752[430], im753[0], im754[0], im755[0], im756[0], im757[0], im758[0], im759[0], im760[0], im761[0], im762[0], im763[0], im764[0], im765[0], im766[0], im767[10], im768[140], im769[180], im770[0], im771[0], im772[0], im773[0], im774[0], im775[0], im776[180], im777[0], im778[10], im779[0], im780[0], im781[0], im782[0], im783[10], im784[10], im785[150], im786[170], im787[60], im788[110], im789[40], im790[420], im791[10], im792[60], im793[1750], im794[10], im795[450], im796[0], im797[0], im798[0], im799[0], im800[0], im801[0], im802[0], im803[580], im804[0], im805[0], im806[0], im807[280], im808[0], im809[0], im810[0], im811[150], im812[240], im813[0], im814[0], im815[20], im816[10], im817[0], im818[0], im819[0], im820[0], im821[0], im822[0], im823[0], im824[0], im825[30], im826[20], im827[10], im828[1050], im829[0], im830[20], im831[0], im832[20], im833[0], im834[0], im835[0], im836[30], im837[0], im838[0], im839[0], im840[0], im841[10], im842[0], im843[0], im844[0], im845[0], im846[0], im847[0], im848[0], im849[10], im850[20], im851[20], im852[0], im853[10], im854[30], im855[10], im856[0], im857[40], im858[0], im859[40], im860[0], im861[0], im862[0], im863[0], im864[0], im865[10], im866[460], im867[0], im868[0], im869[10], im870[0], im871[1790], im872[540], im873[0], im874[0], im875[0], im876[110], im877[170], im878[0], im879[10], im880[0], im881[210], im882[10], im883[150], im884[0], im885[10], im886[170], im887[40], im888[60], im889[0], im890[0], im891[50], im892[0], im893[10], im894[0], im895[300], im896[0], im897[0], im898[0], im899[80], im900[130], im901[10], im902[0], im903[0], im904[0], im905[0], im906[30], im907[0], im908[30], im909[0], im910[0], im911[110], im912[0], im913[0], im914[0], im915[0], im916[0], im917[0], im918[0], im919[0], im920[120], im921[0], im922[10], im923[0], im924[0], im925[10], im926[140], im927[10], im928[0], im929[0], im930[20], im931[0], im932[0], im933[0], im934[10], im935[0], im936[220], im937[220], im938[0], im939[60], im940[40], im941[0], im942[10], im943[0], im944[240], im945[210], im946[0], im947[0], im948[0], im949[20], im950[0], im951[0], im952[10], im953[1130], im954[0], im955[0], im956[0], im957[60], im958[0], im959[0], im960[0], im961[0], im962[0], im963[0], im964[130], im965[40], im966[0], im967[0], im968[230], im969[0], im970[0], im971[0], im972[10], im973[0], im974[0], im975[0], im976[10], im977[0], im978[0], im979[0], im980[10], im981[0], im982[140], im983[0], im984[570], im985[100], im986[0], im987[0], im988[0], im989[0], im990[0], im991[280], im992[20], im993[0], im994[0], im995[0], im996[0], im997[0], im998[0], im999[690]]	
Health sum:100000	

Con 10000:

	StartPause and checkResume num. of immortals: 10000 STOP
im928[3100], im9284[30], im9285[0], im9286[0], im9287[0], im9288[130], im9289[100], im9290[0], im9291[110], im9292[110], im9293[130], im9294[0], im9295[100], im9296[130], im9297[100], im9298[110], im9299[100], im9300[130], im9301[0], im9302[100], im9303[100], im9304[100], im9305[0], im9306[0], im9307[100], im9308[100], im9309[150], im9310[150], im9311[100], im9312[100], im9313[100], im9314[100], im9315[0], im9316[100], im9317[0], im9318[0], im9319[0], im9320[100], im9321[100], im9322[0], im9323[100], im9324[130], im9325[0], im9326[100], im9327[100], im9328[0], im9329[0], im9330[130], im9331[100], im9332[100], im9333[0], im9334[0], im9335[100], im9336[0], im9337[0], im9338[10], im9339[30], im9340[0], im9341[0], im9342[100], im9343[100], im9344[100], im9345[0], im9346[0], im9347[0], im9348[0], im9349[0], im9350[0], im9351[0], im9352[10], im9353[10], im9354[100], im9355[100], im9356[100], im9357[0], im9358[100], im9359[100], im9360[100], im9361[80], im9362[110], im9363[0], im9364[0], im9365[100], im9366[0], im9367[100], im9368[0], im9369[0], im9370[100], im9371[100], im9372[100], im9373[160], im9374[0], im9375[110], im9376[210], im9377[0], im9378[20], im9379[0], im9380[110], im9381[130], im9382[240], im9383[0], im9384[0], im9385[140], im9386[120], im9387[120], im9388[0], im9389[10], im9390[0], im9391[0], im9392[10], im9393[10], im9394[110], im9395[110], im9396[110], im9397[110], im9398[120], im9399[110], im9400[110], im9401[100], im9402[120], im9403[0], im9404[0], im9405[120], im9406[150], im9407[120], im9408[150], im9409[20], im9410[0], im9411[0], im9412[140], im9413[0], im9414[150], im9415[10], im9416[10], im9417[0], im9418[110], im9419[110], im9420[0], im9421[10], im9422[60], im9423[110], im9424[120], im9425[100], im9426[110], im9427[10], im9428[0], im9429[110], im9430[110], im9431[120], im9432[110], im9433[110], im9434[20], im9435[380], im9436[0], im9437[110], im9438[120], im9439[110], im9440[0], im9441[10], im9442[110], im9443[110], im9444[110], im9445[60], im9446[110], im9447[110], im9448[110], im9449[100], im9450[110], im9451[0], im9452[10], im9453[0], im9454[130], im9455[10], im9456[10], im9457[10], im9458[0], im9459[0], im9460[110], im9461[0], im9462[10], im9463[10], im9464[10], im9465[10], im9466[0], im9467[0], im9468[110], im9469[0], im9470[0], im9471[50], im9472[0], im9473[0], im9474[150], im9475[0], im9476[0], im9477[380], im9478[0], im9479[380], im9480[110], im9481[110], im9482[10], im9483[0], im9484[100], im9485[100], im9486[100], im9487[100], im9488[0], im9489[100], im9490[100], im9491[100], im9492[100], im9493[0], im9494[100], im9495[100], im9496[0], im9497[100], im9498[100], im9499[100], im9500[20], im9501[0], im9502[0], im9503[10], im9504[0], im9505[100], im9506[100], im9507[100], im9508[100], im9509[100], im9510[100], im9511[0], im9512[100], im9513[0], im9514[0], im9515[0], im9516[0], im9517[0], im9518[100], im9519[0], im9520[0], im9521[0], im9522[100], im9523[20], im9524[0], im9525[0], im9526[0], im9527[100], im9528[100], im9529[0], im9530[0], im9531[100], im9532[210], im9533[100], im9534[100], im9535[0], im9536[0], im9537[100], im9538[70], im9539[0], im9540[100], im9541[100], im9542[100], im9543[100], im9544[100], im9545[100], im9546[100], im9547[100], im9548[100], im9549[0], im9550[0], im9551[0], im9552[0], im9553[0], im9554[0], im9555[0], im9556[100], im9557[100], im9558[100], im9559[0], im9560[0], im9561[120], im9562[120], im9563[30], im9564[120], im9565[120], im9566[100], im9567[0], im9568[0], im9569[100], im9570[100], im9571[0], im9572[100], im9573[30], im9574[20], im9575[120], im9576[20], im9577[10], im9578[120], im9579[20], im9580[120], im9581[30], im9582[120], im9583[100], im9584[100], im9585[10], im9586[140], im9587[20], im9588[120], im9589[100], im9590[100], im9591[100], im9592[0], im9593[100], im9594[100], im9595[0], im9596[0], im9597[100], im9598[100], im9599[0], im9600[0], im9601[0], im9602[0], im9603[0], im9604[100], im9605[100], im9606[100], im9607[0], im9608[0], im9609[0], im9610[0], im9611[0], im9612[0], im9613[0], im9614[0], im9615[0], im9616[100], im9617[100], im9618[100], im9619[0], im9620[100], im9621[100], im9622[100], im9623[100], im9624[0], im9625[100], im9626[0], im9627[0], im9628[10], im9629[0], im9630[0], im9631[0], im9632[100], im9633[20], im9634[100], im9635[90], im9636[100], im9637[100], im9638[0], im9639[0], im9640[100], im9641[0], im9642[40], im9643[100], im9644[0], im9645[20], im9646[0], im9647[10], im9648[0], im9649[0], im9650[0], im9651[0], im9652[100], im9653[100], im9654[100], im9655[100], im9656[100], im9657[100], im9658[0], im9659[0], im9660[0], im9661[0], im9662[100], im9663[0], im9664[0], im9665[0], im9666[0], im9667[0], im9668[100], im9669[100], im9670[0], im9671[100], im9672[0], im9673[100], im9674[0], im9675[50], im9676[100], im9677[0], im9678[0], im9679[100], im9680[0], im9681[0], im9682[100], im9683[0], im9684[100], im9685[0], im9686[100], im9687[100], im9688[100], im9689[100], im9690[100], im9691[0], im9692[0], im9693[0], im9694[100], im9695[0], im9696[0], im9697[100], im9698[100], im9699[0], im9700[100], im9701[0], im9702[110], im9703[0], im9704[100], im9705[100], im9706[60], im9707[0], im9708[100], im9709[100], im9710[0], im9711[30], im9712[100], im9713[60], im9714[0], im9715[0], im9716[10], im9717[0], im9718[10], im9719[10], im9720[0], im9721[110], im9722[10], im9723[10], im9724[110], im9725[0], im9726[0], im9727[110], im9728[120], im9729[120], im9730[20], im9731[120], im9732[120], im9733[120], im9734[120], im9735[0], im9736[120], im9737[80], im9738[0], im9739[0], im9740[10], im9741[140], im9742[260], im9743[100], im9744[110], im9745[140], im9746[0], im9747[0], im9748[0], im9749[70], im9750[150], im9751[280], im9752[0], im9753[100], im9754[20], im9755[0], im9756[0], im9757[0], im9758[0], im9759[10], im9760[280], im9761[110], im9762[10], im9763[120], im9764[100], im9765[170], im9766[450], im9767[0], im9768[170], im9769[160], im9770[30], im9771[100], im9772[160], im9773[0], im9774[0], im9775[560], im9776[580], im9777[120], im9778[140], im9779[0], im9780[270], im9781[370], im9782[410], im9783[290], im9784[110], im9785[280], im9786[140], im9787[0], im9788[0], im9789[10], im9790[100], im9791[0], im9792[100], im9793[100], im9794[0], im9795[100], im9796[110], im9797[110], im9798[0], im9799[100], im9800[0], im9801[100], im9802[100], im9803[0], im9804[0], im9805[0], im9806[100], im9807[100], im9808[0], im9809[0], im9810[0], im9811[0], im9812[0], im9813[0], im9814[100], im9815[100], im9816[100], im9817[40], im9818[100], im9819[0], im9820[0], im9821[0], im9822[100], im9823[0], im9824[10], im9825[0], im9826[100], im9827[10], im9828[0], im9829[0], im9830[10], im9831[0], im9832[100], im9833[0], im9834[100], im9835[100], im9836[0], im9837[0], im9838[100], im9839[100], im9840[100], im9841[0], im9842[100], im9843[0], im9844[100], im9845[0], im9846[0], im9847[100], im9848[100], im9849[0], im9850[0], im9851[0], im9852[70], im9853[0], im9854[100], im9855[100], im9856[0], im9857[0], im9858[0], im9859[0], im9860[100], im9861[0], im9862[100], im9863[0], im9864[110], im9865[100], im9866[0], im9867[0], im9868[0], im9869[100], im9870[0], im9871[0], im9872[110], im9873[0], im9874[0], im9875[0], im9876[0], im9877[0], im9878[100], im9879[0], im9880[100], im9881[100], im9882[10], im9883[0], im9884[0], im9885[10], im9886[0], im9887[0], im9888[0], im9889[10], im9890[0], im9891[0], im9892[0], im9893[0], im9894[0], im9895[100], im9896[0], im9897[0], im9898[0], im9899[0], im9900[0], im9901[0], im9902[20], im9903[100], im9904[100], im9905[0], im9906[100], im9907[100], im9908[100], im9909[0], im9910[0], im9911[0], im9912[100], im9913[0], im9914[100], im9915[0], im9916[100], im9917[100], im9918[0], im9919[0], im9920[20], im9921[100], im9922[0], im9923[100], im9924[100], im9925[100], im9926[100], im9927[100], im9928[0], im9929[0], im9930[0], im9931[0], im9932[10], im9933[0], im9934[0], im9935[100], im9936[0], im9937[100], im9938[100], im9939[100], im9940[100], im9941[100], im9942[100], im9943[100], im9944[100], im9945[100], im9946[280], im9947[100], im9948[110], im9949[110], im9950[160], im9951[160], im9952[160], im9953[160], im9954[160], im9955[160], im9956[160], im9957[160], im9958[160], im9959[160], im9960[160], im9961[160], im9962[160], im9963[160], im9964[160], im9965[160], im9966[160], im9967[160], im9968[160], im9969[160], im9970[160], im9971[160], im9972[160], im9973[160], im9974[160], im9975[160], im9976[160], im9977[160], im9978[160], im9979[160], im9980[160], im9981[160], im9982[160], im9983[160], im9984[160], im9985[160], im9986[160], im9987[160], im9988[160], im9989[160], im9990[160], im9991[160], im9992[160], im9993[160], im9994[160], im9995[160], im9996[160], im9997[160], im9998[160], im9999[160], im10000[160]	

check"/"Resume", confirmando que el invariante se cumple de forma robusta incluso a gran escala.

10. Un elemento molesto para la simulación es que en cierto punto de la misma hay pocos 'inmortales' vivos realizando peleas fallidas con 'inmortales' ya muertos. Es necesario ir suprimiendo los inmortales muertos de la simulación a medida que van muriendo. Para esto:

Analizando el esquema de funcionamiento de la simulación, esto podría crear una condición de carrera? Implemente la funcionalidad, ejecute la simulación y observe qué problema se presenta cuando hay muchos 'inmortales' en la misma. Escriba sus conclusiones al respecto en el archivo RESPUESTAS.txt.

Corrija el problema anterior SIN hacer uso de sincronización, pues volver secuencial el acceso a la lista compartida de inmortales haría extremadamente lenta la simulación.

Respuestas y solución de este punto lo dejamos en RESPUESTAS.txt según lo solicitado en el enunciado.

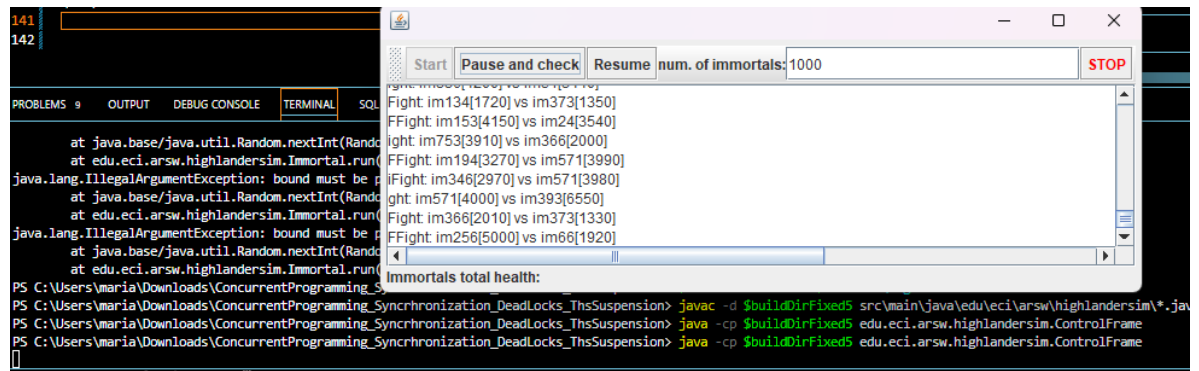
Capturas de antes del arreglo:

```
Exception in thread "Awt-EventQueue-0" java.util.ConcurrentModificationException
```

```
Exception in thread "im80" java.lang.NullPointerException: Cannot read field "prev" because "x" is null
    at java.base/java.util.LinkedList.node(LinkedList.java:588)
    at java.base/java.util.LinkedList.get(LinkedList.java:488)
    at edu.eci.arsw.highlandersim.Immortal.run(Immortal.java:50)
```

```
java.lang.IllegalArgumentException: bound must be positive
    at java.base/java.util.Random.nextInt(Random.java:557)
    at edu.eci.arsw.highlandersim.Immortal.run(Immortal.java:43)
java.lang.IllegalArgumentException: bound must be positive
    at java.base/java.util.Random.nextInt(Random.java:557)
    at edu.eci.arsw.highlandersim.Immortal.run(Immortal.java:43)
java.lang.IllegalArgumentException: bound must be positive
    at java.base/java.util.Random.nextInt(Random.java:557)
    at edu.eci.arsw.highlandersim.Immortal.run(Immortal.java:43)
```

Luego del arreglo: Sin problemas ni excepciones:

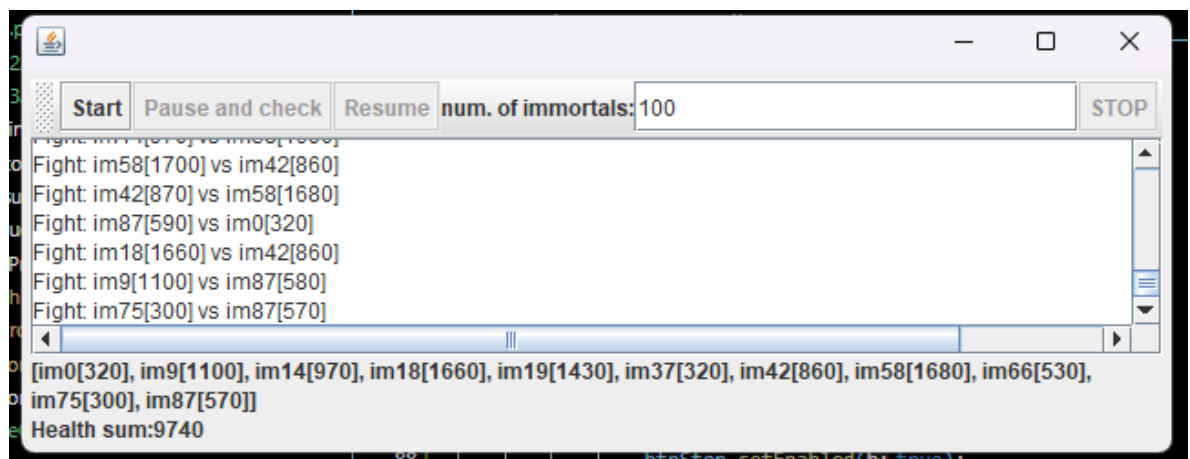


11. Para finalizar, implemente la opción STOP.

Implementamos el botón STOP siguiendo el mismo patrón cooperativo usado en pause/resume (bandera + wait/notify, sin Thread.stop(), que también está deprecado por razones similares a suspend()/resume()). Cada Immortal tiene una bandera stopped que revisa al inicio de cada vuelta de su ciclo; al activarla, el hilo termina su ejecución de forma limpia (incluso si estaba pausado en ese momento, se despierta para poder detenerse). El botón STOP también deshabilita "Pause and check"/"Resume" y vuelve a habilitar "Start", permitiendo reiniciar la simulación desde cero.

Probamos STOP en distintos escenarios, con la simulación corriendo normalmente, en medio de una pausa, y con distintos números de inmortales y en todos los casos los hilos terminan correctamente sin quedar "zombies" corriendo en segundo plano. Además, tras un STOP, "Start" permite arrancar una simulación completamente nueva (salud reiniciada en 100 para todos), confirmando que el ciclo detener/reiniciar funciona de forma consistente.

Dejamos correr unos segundos y pusimos STOP:



Luego lo iniciamos otra vez y efectivamente se inicia de nuevo:

